Rétro-ingénierie

Introduction Windows Internals

Airbus CERT

Table des matières

1 Introduction 3
2 C'est quoi le Reverse Engineering ? 5
3 Motivations 7
4 Le format PE (Portable Executable) 23
5 Méthodologie d'analyse 43
6 La sécurité logicielle 66
7 Drivers Windows 77
8 Cas d'usage : EDR 92
9 BYOVD 103
10 Patch diffing 110
11 Conclusion 118

1 Introduction

Objectifs de ce cours

- C'est quoi le Reverse Engineering ?
- Motivation
- Découverte des Windows Internals
- Introduction au développement de drivers
- Exploitation de Windows par des drivers vulnérables
- Présentation de mécanismes de sécurité propres à Windows
- Introduction au patch diffing

Posez des questions !

Pourquoi Windows ?

- Grande utilisation en milieu entreprise et particulier
- Active Directory
- Cible privilégiée pour le développement de malware

2 C’est quoi le Reverse Engineering ?

Les grands principes

- Analyse en boîte noire d'un système en vue d'en comprendre le fonctionnement interne
- Idée de reconstruire une représentation dans un niveau d'abstraction supérieur
- IT sec : traditionnellement appliqué sur du code binaire

3 Motivations

Un brève histoire de la sécurité

- Histoire
- Défense
- Attaque
- Entre les deux

Histoire

Dans les années 80/90s

- Cracking et demoscene Amiga[1], Apple II[2], Atari, puis DOS, etc.
- Virus informatique[3]

[1] http://www.pouet.net/

[2] https://www.filfre.net/2015/12/a-pirates-life-for-me-part-1-dont-copy-that-floppy/

[3] https://archive.org/details/malwaremuseum&tab=collection

Amiga Scene[4]



Fig. 1. – Amiga scene

Crack Templates

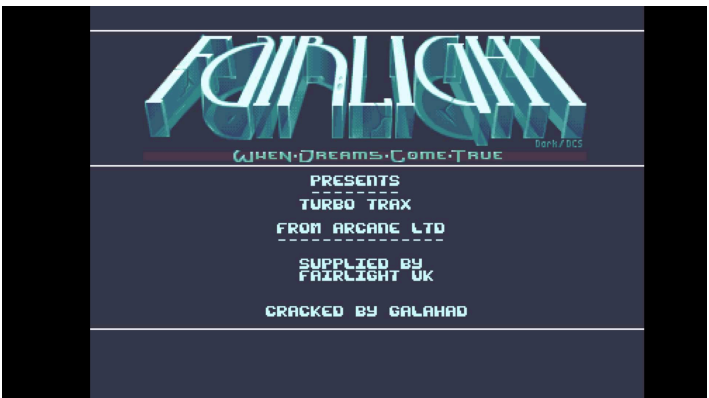


Fig. 2. – Crack templates

Keygen



Fig. 3. – keygen

Fin 90/début 2000

- Cracking scene jeux/shareware
  - Écosytème des shareware Windows
  - Développement des outils (debuggers, patcher)
  - Développement des compressors/packers/unpackers (AsProtect, Armadillo, etc.)
  - Exemples de protections : SecuROM, SafeDisc, StarForce
- Mais aussi hack/exploit (monde Unix)
  - Phrack[5]
  - Smashing The Stack For Fun And Profit, Aleph One
  - The Last Stage of Delirium[6] (LSD)

[5] http://www.phrack.org/
[6] http://lsd-pl.net/

Un virage en 2009 : Opération Aurora[7]

- Piratage de Google
  - Attribué à la Chine
  - Espionnage des comptes Gmail
  - Espionnage industriel
  - Incident diplomatique, retrait de Google de Chine

Les années 2010

- Affaire Snowden (2013)[8], prise de conscience des capacités offensives des Etats-Unis
- Loi de programmation militaire (2014)
- Google Project Zero (2014)[9]
- EternalBlue (MS17-010)
- Structuration du secteur
  - Formation
  - Rôle de la sécurité informatique dans l'organigramme de l'entreprise



[8] https://fr.wikipedia.org/wiki/R%C3%A9v%C3%A9lations_d%27Edward_Snowden
[9] https://projectzero.google/

Les années 2020

- Apparition/Disparition des menaces Ransomware
- IA



Défense (aka blue team)

- Entités
  - Security Operation Center SOC
  - Detection Engineering
  - Computer Emergency Response Team (CERT)[10]
- Missions
  - Incidents
  - Sensibilisation
  - Analyse de malware, exploits, incidents, etc.
  - Forensic/investigation numérique

[10] https://www.ssi.gouv.fr/agence/cybersecurite/ssi-en-france/les-cert-francais/

Défense (aka blue team)

- *C'est nous!*
  - Équipe interne de réponse à incident pour le groupe Airbus
  - Airbus, suppliers, affiliates
  - Publis : https://skyblue.team et outils : https://github.com/airbus-cert



Attaque (aka red team)

- Entités
  - Très nombreuses boîtes de services/pentests/Red Team
  - Boîtes d'intelligence économique
  - Étatiques DGSE[11], DGSI, DGA, etc...
- Missions
  - Pentest/outillage
  - Recherche de vulnérabilité
  - Développement d'exploit
  - Lutte Informatique Offensive LIO[12][13]

[11] https://www.sstic.org/2018/presentation/2018_cloture/

[12] http://www.livreblancdefenseetsecurite.gouv.fr/

[13] https://www.senat.fr/rap/r07-449/r07-4496.html

Entre les deux : Anti Cheat

- Industrie du Jeux Video
  - Virtual goods & in-game economy : $40B[14]
  - 3.3B de joueurs à travers le monde
- Menaces
  - DRM Bypass
  - Triche
  - Fraude
  - Vol de comptes

[14] Keynote Phillipe Paquet SSTIC 2026 : https://www.sstic.org/2026/presentation/invite_2026_1/

Entre les deux : Anti Cheat

'Operation Chicken Drumstick': How Chinese police cracked the world's biggest video game cheat syndicate

By Dong Xing

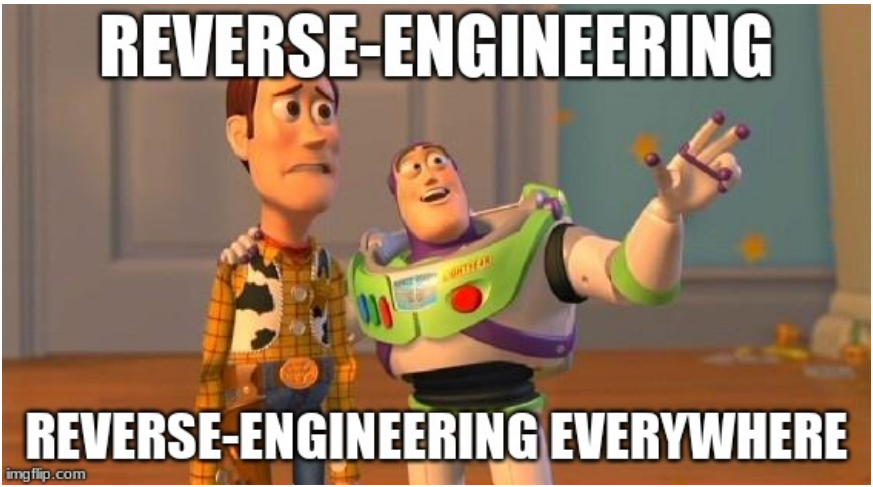
Arts, Culture and Entertainment

Posted 1 Apr 2021 at 9:09pm, updated 15 Jul 2021 at 3:31pm



Police say they seized these cars from the group's "global financial leader" as they arrested him at his residence. *(Supplied: Kunshan Public Security Bureau)*

Entre les deux : Anti Cheat



4 Le format PE (Portable Executable)

Introduction

- Matière première
  - Programmes informatiques distribués sous forme d'exécutables
  - Code source ⟶ chaîne de compilation ⟶ exécutable
- Objectifs
  - Quelques bases sur le format `PE` (Windows)
  - Concepts similaires dans le format `ELF`
  - Transition fichier sur le système de fichiers vers un programme s'exécutant en mémoire

Structure du format PE

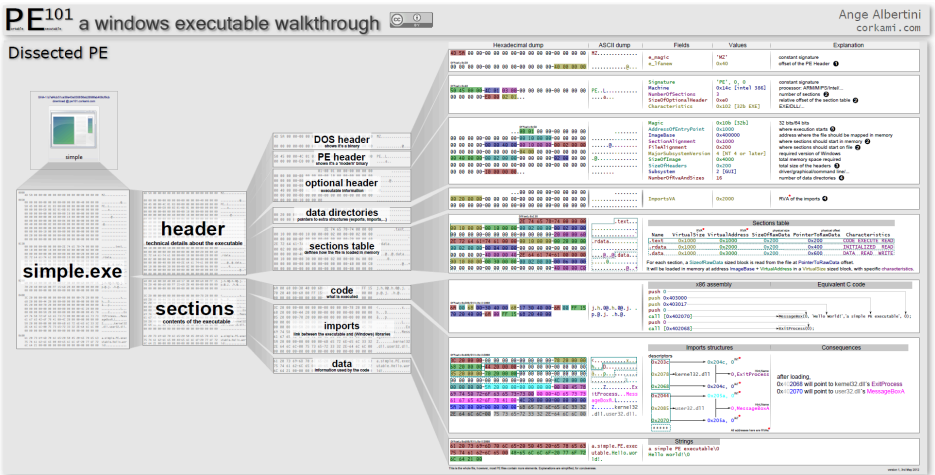
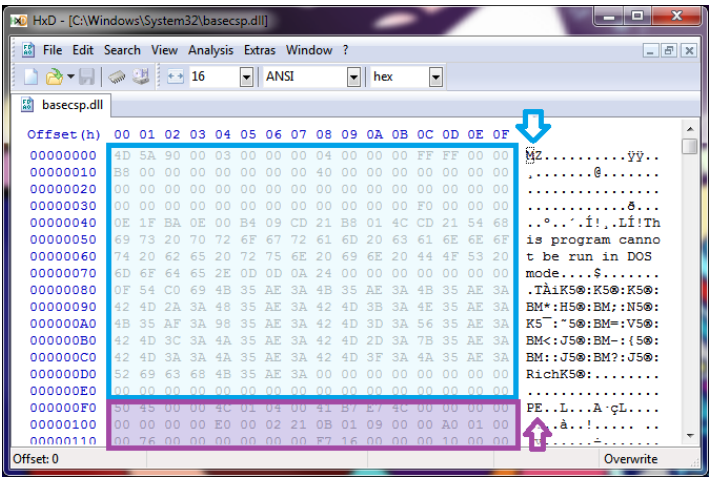


Fig. 9. – Récapitulatif détaillé : OpenRCE's Portable Executable Format Layout

Portable executable file format

- Format des fichiers exécutables : le format `PE`
- Introduit par `Microsoft` en même temps que Windows NT 3.1
- Initialement conçu à partir du format `UNIX COFF`
- L'extension 64-bit s'appelle `PE+`
- Deux signatures incontournables : `MZ` et `PE`:
  - Le format `PE` débute par un en-tête DOS, conservé uniquement pour des raisons de compatibilité.
  - Il est suivi par l'en-tête *utile* : le *PE header*.
- L'en-tête `PE` contient toutes les informations destinées au **chargeur dynamique d'exécutable** (*loader*).

En-tête DOS


HxD - [C:\Windows\System32\basecsp.dll]
File Edit Search View Analysis Extras Window ?
16
ANSI
hex
basecsp.dll
Offset (h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00000000 4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00
00000010 B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00
00000020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000030 00 00 00 00 00 00 00 00 00 00 00 00 F0 00 00 00
00000040 0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68
00000050 69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F
00000060 74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20
00000070 6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00
00000080 0F 54 C0 69 4B 35 AE 3A 4B 35 AE 3A 4B 35 AE 3A
00000090 42 4D 2A 3A 48 35 AE 3A 42 4D 3B 3A 4E 35 AE 3A
000000A0 4B 35 AF 3A 98 35 AE 3A 42 4D 3D 3A 56 35 AE 3A
000000B0 42 4D 3C 3A 4A 35 AE 3A 42 4D 2D 3A 7B 35 AE 3A
000000C0 42 4D 3A 3A 4A 35 AE 3A 42 4D 3F 3A 4A 35 AE 3A
000000D0 52 69 63 68 4B 35 AE 3A 00 00 00 00 00 00 00 00
000000E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000000F0 50 45 00 00 4C 01 04 00 41 B7 E7 4C 00 00 00 00
00000100 00 00 00 00 E0 00 02 21 0B 01 09 00 00 A0 01 00
00000110 00 76 00 00 00 00 00 00 F7 16 00 00 00 10 00 00
Offset: 0
Overwrite

En-tête PE

L'en-tête `PE` de type `IMAGE_NT_HEADER` référence un champ `IMAGE_OPTIONAL_HEADER` qui n'a rien d'optionnel car la plupart des informations essentielles y sont stockées, telles que :

- Le point d'entrée : adresse de la première instruction.
- L'Image Base : adresse favorite à laquelle le binaire sera chargé en mémoire.
- La taille du code.
- Une structure `IMAGE_DATA_DIRECTORY`, elle aussi très importante pour les imports, exports, relocations …

Les informations à propos des sections sont stockées de l'en-tête `PE` dans la structure `IMAGE_FILE_HEADER`.

Théorie de la relativité

> *If the adress is relative to something, adding that something to it will remove the relativity.*
>
> — Tutoriel, Lord Julus, 1998

Point sémantique

- Adresse virtuelle (**VA**) : désigne l'adresse en mémoire.
- *Image base* : **VA** à laquelle est chargé (mappé) le binaire.
- Adresse virtuelle relative (**RVA**) : adresse virtuelle relative à l'*image base*.

Traité de mathématiques appliquées

- La plupart des adresses présentes dans l'en-tête `PE` sont des **RVA**.
- $\mathbf{VA} = \mathbf{RVA} + ImageBase$

Les sections

- Une section est un conteneur logique.
- Elle est décrite par un `IMAGE_SECTION_HEADER`.
- Un jeu de flags spécifie ses propriétés : `R/W/X` …
- Elle est identifiée par un nom, le plus souvent lié au type de données qu'elle contient :
  - `.CODE` ou `.text` : segments de code, le point d'entrée se situe dans cette section (*a priori*).
  - `.bss` : données non initialisées, sa taille sur le disque dur est nulle.
  - `.data` : variables globales.
  - `.rsrc` : contient les ressources (icônes, images, …).
  - etc.

La convention de nommage n'est donnée qu'à titre indicatif, néanmoins la majorité des compilateurs respectent ces conventions.

La table d'imports

Tout binaire interagit d'une manière ou d'une autre avec le système d'exploitation. Pour cela il utilise l'API fournie par ce dernier.

- Sur un noyau Windows cette interface a été découpée en bibliothèques (*dynamic-link library* ou DLL) comme `kernel32.dll`.
- La table d'imports permet au binaire de spécifier les fonctions dont il a besoin. Pour cela il spécifie le nom de la bibliothèque et de chacune des fonctions requises.
- Dans l'en-tête PE, les imports sont décrits à l'aide de la structure `IMAGE_DIRECTORY_ENTRY_IMPORT`.

La table d'exports

Un binaire peut souhaiter rendre disponible une partie de son code à d'autres binaires. Ce cas de figure concerne avant tout les bibliothèques qui sont alors compilées sous forme de DLL.

Une interface dédiée est utilisée : la table d'exports, décrite à l'aide de la structure `IMAGE_DIRECTORY_ENTRY_EXPORT`.

- Deux possibilités pour exporter une fonction :
  - par nom, comme `kernel32.dll` qui exporte `ExitProcess`.
  - par ordinal : un entier (16-bits) identifie la fonction exportée au sein de la `DLL` de manière unique.
- La structure `IMAGE_EXPORT_DIRECTORY` permet de faire le lien entre une fonction exportée et son adresse dans le binaire.

La table de relocations

Un binaire ne va pas connaître à l'avance l'adresse de base à laquelle il va être chargé. Certaines instructions sont dépendantes de cette position et doivent être recalculées lors du chargement. C'est ce que l'on appelle « relocation ».

Il faut le voir comme un ensemble d'instructions de relocation qui seront appliquées lors du chargement en mémoire. Cette ensemble d'instructions est stocké dans la *table de relocation*, décrite à l'aide de la structure `IMAGE_DIRECTORY_ENTRY_BASERELOC`.

Le malware Relock utilisait cette table en tant que packer. En effet, sa section de code était vide, sa table de relocation contenait un ensemble d'instructions permettant de reconstruire le code du malware. Il utilisait aussi un bug dans Windows qui, quand l'adresse de base demandée par le programme était « 0 », chargeait le programme automatiquement à 0x40000, ce qui constituait un détournement de l'ASLR.

Mini TP PE : découverte du format

`https://tls-sec.skyblue.team/mini.html`

Pour chaque binaire, à l'aide de Detect-it-Easy:

- noter l'architecture cible
- retrouver la version du *linker* et la date de compilation
- trouver la section la plus grosse: 1) sur le disque, 2) en mémoire
- noter un nom de DLL importée et un nom de fonction importée

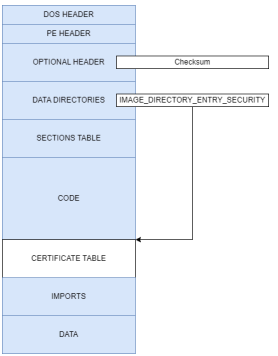
Les protections autour du binaire Windows

Signature

- Sous Windows, on ne distingue pas les exécutables et les bibliothèques, on parle de *module*
- Il est possible de signer chaque module afin d'être verifié lors du chargement
- La signature se calcule en excluant
  - Le champ `checksum` présent dans `IMAGE_OPTIONAL_HEADER`
  - L'entrée `IMAGE_DIRECTORY_ENTRY_SECURITY` des `DATA DIRECTORIES`, qui contient l'adresse de la table des certificats
  - Et enfin la table des certificats définie dans la `CERTIFICATE TABLE` précédente

Les protections autour du binaire Windows

- Le fait d'exclure des parties du PE lors du calcule amène naturellement une zone d'ombre
- C'est l'objet de SigFlip.exe[15][16]



[15] SigFlip : https://github.com/med0x2e/SigFlip

[16] Blackhat : https://www.blackhat.com/docs/us-16/materials/us-16-Nipravsky-Certificate-Bypass-Hiding-And-Executing-Malware-From-A-Digitally-Signed-Executable-wp.pdf

Le loader Windows

Description du loader de binaire

- Composant essentiel et pourtant son utilisation est transparente : appel implicite lors du lancement d'un exécutable.
- Utilise les informations contenues dans l'en-tête PE pour assurer le chargement en mémoire du binaire.
- Responsable de la création des objets internes à l'OS.
  - Exemple : les objets *Process*, *Thread*.
- Notifie le système de la création et démarre le thread principal.

Chargement des exécutables en mémoire

- Un processus ⇒ un espace d'adressage virtuel indépendant (cf. `CR3`)
- Projection dans cet espace mémoire : ce n'est pas une simple copie.
  - Mapping des sections
  - Création de zones mémoire avec les attributs correspondants
  - Exemple : la section `BSS` (variables non initialisées).

Le loader Windows

- Le binaire est chargé à une adresse préférentielle : l'*image base*. Historiquement `0x400000` (mais `PIE`/`ASLR` peuvent le changer)
- Si le binaire n'a pu être chargé à son adresse préférentielle : utilisation des informations de la section `.reloc`.

Chargement des modules

Le loader charge toutes les bibliothèques nécessaires[17] dans l'espace du processus :

- Il consulte la table des imports du processus en cours de création.
- Pour chaque DLL nécessaire, il la place dans l'espace d'adressage du processus et consulte sa table d'exports afin de déterminer les adresses des fonctions importées.

[17] Pour aller plus loin: https://docs.microsoft.com/en-us/windows/desktop/dlls/dynamic-link-library-search-order

Le loader Windows

- Il assigne alors les champs de la table d'imports du processus avec ces adresses. Cette zone est appelée l'*import address table* (`IAT`).

Linux/ELF en résumé

Assez proche d'un PE sur le fond:

- le *file header* contient les informations critiques : architecture cible, *endianness*, point d'entrée, etc.
- les *program headers* définissent l'interpréteur (`ld.so`) et la cartographie mémoire,
- les *section headers* (optionnels) définissent les informations supplémentaires: bibliothèques à charger, symboles externes, relocations, etc.

```
$ ldd /bin/ls
linux-vdso.so.1 (0x00007ffdd65cb000)
libselinux.so.1 => /lib/x86_64-linux-gnu/libselinux.so.1
(0x00007fa617bd1000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
(0x00007fa617832000)
```

Linux/ELF en résumé


```
libpcre.so.3 => /lib/x86_64-linux-gnu/libpcre.so.3
(0x00007fa6175bf000)
libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2
(0x00007fa6173bb000)
/lib64/ld-linux-x86-64.so.2 (0x00007fa61801a000)
libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0
(0x00007fa61719e000)
```

Commande pour regarder un ELF: `readelf -a`

Quelques différences:

- Concept d'*interpréteur*:
  - Si `PT_INTERP` existe, le noyau charge l'exécutable pointé par cette entrée. Il est chargé de gérer lui-même la résolution des bibliothèques dynamiques.

Linux/ELF en résumé

  - Sinon, le noyau mappe les sections et jump au point d'entrée du ELF directement.
- Un exécutable statique ne dépend d'aucune bibliothèque externe: les appels système se font au travers de `syscall`.
  - Impossible sous Windows: les `syscall` ne sont pas stables d'une version à l'autre. On passe par des *stubs* dans `kernel32.dll` et autres

5 Méthodologie d’analyse

Les niveaux d'abstractions

- Il existe une multitude d'approches possibles d'un exécutable : les niveaux d'abstractions.
- Depuis le plus bas : le code machine.
- Au plus haut : la *black-box*.
- Dans toute analyse le choix du niveau d'abstraction dépend à la fois de l'échantillon et de ce que l'on cherche à observer.
- Quelques abstractions *haut-niveau* extraites d'un exécutable :
  - Le graphe d'appel.
  - Le *Control Flow Graph* (`CFG`).
  - Le data flow graph.

Le Control Flow Graph

Control transfer instruction **(CTI)**

Les niveaux d'abstractions

C'est une instruction qui peut briser la linéarité du code et transfère le contrôle : un saut, un `RET`, un `CALL` ou une `INT`.

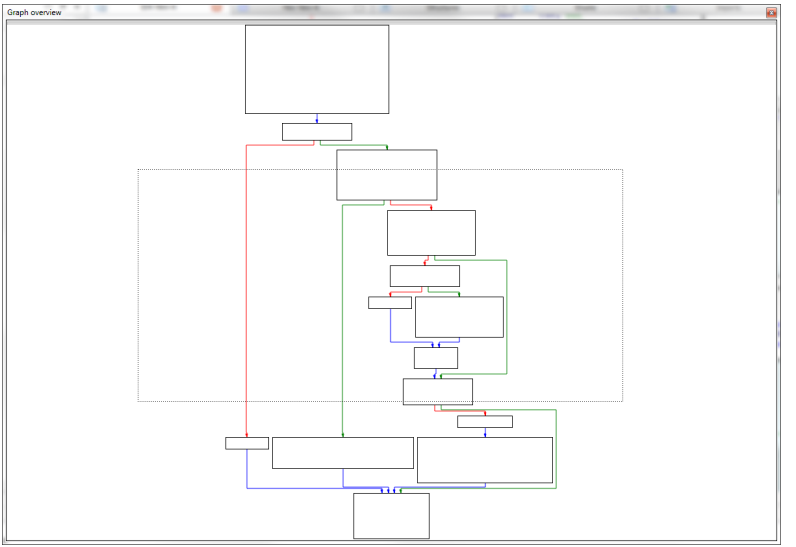
Basic block

Un bloc basique est une séquence contiguë d'instructions ne comportant aucune **CTI** et n'étant la cible d'aucune **CTI**. Seule la première instruction de cette séquence est la cible d'une ou plusieurs **CTI**.

- Le control flow graph (**CFG**) est la combinaison de l'ensemble des basic blocks avec l'ensemble des **CTI**.
- Très intuitif pour comprendre la « logique » d'un programme.
- C'est le mode graphe d'**IDA**.

Les niveaux d'abstractions

CFG, exemple :



Les niveaux d'abstractions

Data flow et graphe d'appel

Data flow

- Chaque nœud représente l'accès en lecture ou en écriture d'une variable
- Backtracking sur les variables pour détecter les zones *sensibles*.
- Analyse combinée du data flow et control flow : *slicing*

Le graphe d'appel

- Permet de visualiser les relations de dépendances.
- Permet d'identifier les fonctions importantes
- Certaines figures sont aussi caractéristiques : machines virtuelles, fonction de chiffrement, etc.

Les niveaux d'abstractions

Les structures conditionnelles

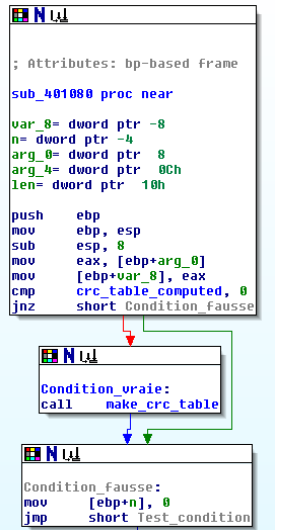
- Comprendre le processus de la génération du code est important
- Faire le lien entre un CFG et les structures de plus hauts niveaux (décompilation mentale).
- L'exemple suivant permet de voir les structures conditionnelles basiques *IF* et *FOR* :

```
if (!crc_table_computed)
    make_crc_table();

for (n = 0; n < len; n++) {
    c = crc_table[(c ^ buf[n]) & 0xff] ^ (c >> 8);
}
```

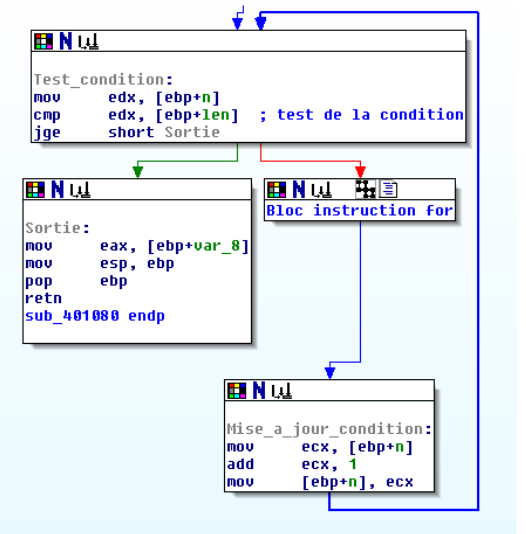
Les niveaux d'abstractions

Structure conditionnelle *IF*



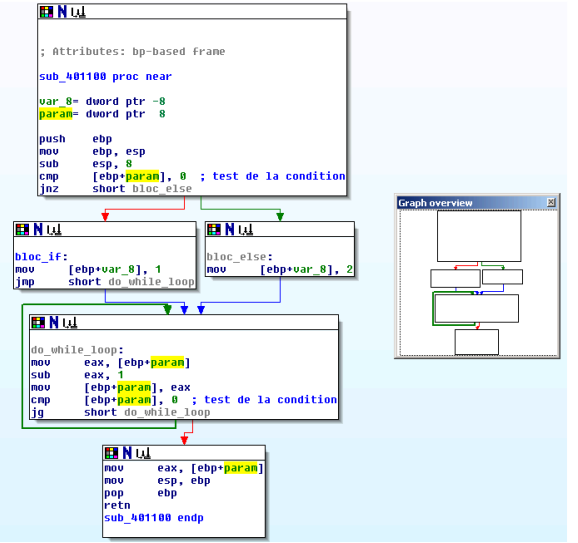
Les niveaux d'abstractions

Structure conditionnelle *FOR*



Les niveaux d'abstractions

Structure conditionnelle *IF THEN ELSE* et *DO WHILE*



L'observation/instrumentation

L'analyse du code est un processus lent, trop coûteux sur de larges portions de code : il est nécessaire de cibler notre analyse.

Que pouvons-nous observer ?

- Le comportement du programme.
  - Exemple : attend-il des entrées de l'utilisateur ?
- Le programme lui-même : nombreuses informations contenues dans l'en-tête `PE`.
- Interaction du programme avec son environnement : accès au système de fichiers, utilisation d'`API`.

L'observation/instrumentation

Observation : approche purement statique

Un binaire peut comporter une multitude d'indices

- Nature du binaire à étudier : exécutable, DLL, driver ou autre
- **Toutes les chaînes de caractères**
- Reconnaissance du compilateur
- Bibliothèques linkées statiquement
- Reconnaissance de packers
- Reconnaissance d'algorithme cryptographique par recherche de constante

L'observation/instrumentation


```
#define A 0x67452301
#define B 0xefcdab89
#define C 0x98badcfe
#define D 0x10325476

void md5Init(MD5Context *ctx){
    ctx->size = (uint64_t)0;

    ctx->buffer[0] = (uint32_t)A;
    ctx->buffer[1] = (uint32_t)B;
    ctx->buffer[2] = (uint32_t)C;
    ctx->buffer[3] = (uint32_t)D;
}
```

L'observation/instrumentation

Cas pratique : message d'erreur/debug

« `OpenSSL 1.0.2o-fips  27 Mar 2018` »

- Indique que le binaire utilise la bibliothèque `OpenSSL`
- L'utilisation des signatures (motifs caractéristiques) correspondantes permettra de reconnaître les fonctions automatiquement : temps d'analyse économisé
- Utilisation possible de chiffrement/RNG/hachage

L'observation/instrumentation

Observation : approche comportementale

- Analyse comportementale
- Caractérisation d'actions suspectes
- API spying, API hooking, p-ex. : framework Detours de Microsoft[18]
- OS instrumentés, sandbox, ex :
  - Cuckoo Sandbox
  - Joe Sandbox

Application à l'analyse de malware, ex. type de comportements suspects :

- Injection de thread.
- Création de fichiers/process.
- Écriture dans un process tiers.
- Accès à la base de registre :
  `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`

[18] http://research.microsoft.com/en-us/projects/detours/

L'observation/instrumentation

API *spying*, API *hooking*

Abus de confiance

À un niveau de privilège `n`, il existe une relation de confiance implicite vers le niveau `n-1` : un programme fait confiance aux **API** qu'il appelle. Le `ring3` fait confiance au `ring0` et ainsi de suite.

- Le binaire est alors vu comme une *black-box*.
- Interception de toutes les **API** utilisées par le binaire :
  - Surveillance des paramètres d'appel et des valeurs de retour : *spying*.
  - Éventuelle modification : *hooking*.
- Techniques plus ou moins intrusives : *dll proxy*, *hook* en `ring3` ou `ring0`, *.local*.
- Permet de conjecturer le comportement global du binaire.

L'observation/instrumentation

Il existe de nombreux framework de **API** *spying*, **API** *hooking*. Sous Windows le plus célèbre est Detours[19]

`Detours` : le *trampoline hooking*

`Detours`[20] est une bibliothèque de Microsoft permettant d'intercepter des appels de fonctions (API Win32) **à l'exécution**, sans recompiler la cible.

Principe : le *inline hooking* par trampoline

- On réécrit les premières instructions (*prologue*) de la fonction cible par un saut (`JMP`) vers notre fonction *detour*.
- Les instructions écrasées sont recopiées dans un *trampoline*, suivi d'un `JMP` vers la suite de la cible.

[19] https://github.com/microsoft/detours
[20] https://github.com/microsoft/detours

L'observation/instrumentation

- Le *detour* peut ainsi appeler l'original via le trampoline : *spying* (observer) ou *hooking* (modifier).

- Fonctionne *in-process* sur `x86`, `x64`, `ARM`, `ARM64`.
- Peut aussi injecter une DLL dans un nouveau process (`DetourCreateProcessWithDllEx`).

Avant le hook

```
TargetFunc:
  push rbp      <-- prologue
  mov  rbp, rsp
  ...           <-- corps
  ret
```

Transaction atomique

Les hooks sont posés dans une transaction qui gèle les threads pour éviter d'exécuter du code à moitié patché.

Après le hook

L'observation/instrumentation


```
TargetFunc:
  jmp  DetourFunc    <--
patché
  ...                <--
corps

Trampoline:
  push rbp           <--
instrs
  mov  rbp, rsp      <--
sauvées
  jmp  TargetFunc+N  <--
retour
```

```
DetourTransactionBegin();
DetourUpdateThread(GetCurrentThread
DetourAttach(&(PVOID&)Real_MessageB

Hook_MessageBoxW);
DetourTransactionCommit();
```

- `Real_*` pointe vers le **trampoline** (l'original).
- `DetourDetach` pour retirer le hook proprement.

Dynamic Binary Instrumentation (DBI)

Modification directe du code executé pour ajouter l'instrumentation

L'observation/instrumentation

- Accès à l'état interne du programme durant son exécution.
- Beaucoup plus performant que le debug
- Granularité possible extrêmement fine : *ex.* log de tous les accès mémoire.
- Exemples :
  - `Pin`[21]
  - `Valgrind`[22]
  - `DynamoRio`[23]
  - `QBDI`[24]

[21] https://software.intel.com/en-us/articles/pin-a-dynamic-binary-instrumentation-tool
[22] http://valgrind.org/
[23] http://www.dynamorio.org/
[24] https://qbdi.quarkslab.com

Analyse de programmes exécutables

- Démarche très coûteuse en temps, d'où la nécessité d'avoir ciblé les zones à analyser.
- Nous nous trouvons à un niveau d'abstraction très bas : le langage assembleur ⇒ Il va falloir remonter, le plus souvent cela passe par la reconstruction du *control-flow-graph*.
- Deux approches possibles :
  - Dynamique à l'aide d'un débogueur
  - Statique grâce à un désassembleur
- Deux approches complémentaires : se servir du meilleur de chacune.

Analyse statique

Avantages

- Vue globale du binaire : étude macroscopique puis microscopique.
- Reconstruction du *control-flow-graph*, *data-flow*.

Analyse de programmes exécutables

- La référence : désassembleur **IDA Pro**.
- Technique de prédilection pour l'étude de code potentiellement malveillant car pas d'exécution.

Limitations

- Difficultés ou impossibilités à prédire la valeur de variables résolues dynamiquement.
- Le désassemblage peut être perturbé (au bas mot) par des techniques d'obfuscation.
- Difficultés avec les codes se déchiffrant en mémoire.

Analyse de programmes exécutables

Analyse dynamique

Avantages

- L'outil de prédilection est le débogueur : **x64dbg**, **WinDbG**, **GDB**, **LLDB**, ...
- Accès direct à toutes les valeurs manipulées par le programme.
- Contrôle du flot d'exécution.
- Plus adaptée à une étude microscopique : mise en place de points d'arrêts et exécution pas-à-pas du code.

Limitations

- Obligation d'exécuter le code : code potentiellement dangereux pour le système d'exploitation hôte.

Analyse de programmes exécutables

- Techniques relativement intrusives : contre-mesures éventuelles visant les outils d'analyse.

6 La sécurité logicielle

Intro

Enfin de la sécurité

- Sécurité hardware
- Sécurité software

Objectifs

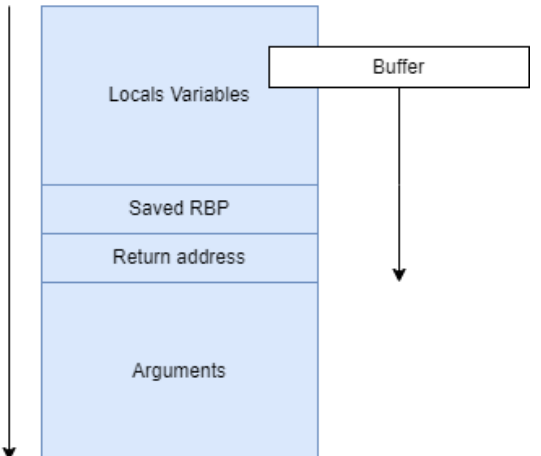
- Comprendre les mécanismes de sécurité afin de les identifier lors de la phase de reverse
- Comprendre les différences entre l'assembleur et la vue décompilée

Canary cookie

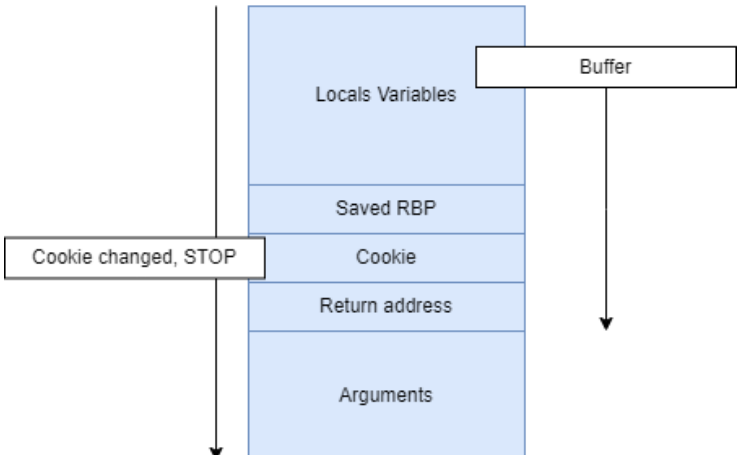
Le `Canary cookie` ou en français ... (pas trouvé dans le CyberDico[25] de l'ANSSI) est un mécanisme de sécurité introduit à la compilation afin de se protéger des catégories d'attaques : `stack overflow`, `return to stack`

[25] CyberDico : https://cyber.gouv.fr/publications/cyberdico-quest-ce-que-cest

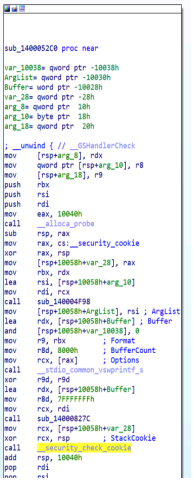
Canary cookie



Canary cookie



Canary cookie



```
sub_1400052C0 proc near

var_10038= qword ptr -10038h
ArgList= qword ptr -10030h
Buffer= word ptr -10028h
var_28= qword ptr -28h
arg_8= qword ptr  10h
arg_10= byte ptr  18h
arg_18= qword ptr  20h

; __unwind { // __GSHandlerCheck
mov     [rsp+arg_8], rdx
mov     qword ptr [rsp+arg_10], r8
mov     [rsp+arg_18], r9
push    rbx
push    rsi
push    rdi
mov     eax, 10040h
call    __alloca_probe
sub     rsp, rax
mov     rax, cs:__security_cookie
xor     rax, rsp
mov     [rsp+10058h+var_28], rax
mov     rbx, rdx
lea     rsi, [rsp+10058h+arg_10]
mov     rdi, rcx
call    sub_140004F9B
mov     [rsp+10058h+ArgList], rsi ; ArgList
lea     rdx, [rsp+10058h+Buffer] ; Buffer
and     [rsp+10058h+var_10038], 0
mov     r9, rbx         ; Format
mov     r8d, 8000h      ; BufferCount
mov     rcx, [rax]      ; Options
call    __stdio_common_vswprintf_s
xor     r9d, r9d
lea     rdx, [rsp+10058h+Buffer]
mov     r8d, 7FFFFFFFh
mov     rcx, rdi
call    sub_14000827C
mov     rcx, [rsp+10058h+var_28]
xor     rcx, rsp        ; StackCookie
call    __security_check_cookie
add     rsp, 10040h
pop     rdi
pop     rsi
```

Canary cookie


```
__int64 sub_1400052C0(__int64 a1, const wchar_t *a2, ...)
{
  unsigned __int64 *v4; // rax
  wchar_t Buffer[32768]; // [rsp+30h] [rbp-10028h] BYREF
  va_list va; // [rsp+10070h] [rbp+18h] BYREF

  va_start(va, a2);
  v4 = (unsigned __int64 *)sub_140004F98();
  _stdio_common_vswprintf_s(*v4, Buffer, 0x8000ui64, a2, 0i64, va);
  return sub_14000827C(a1, Buffer, 0x7FFFFFFFi64, 0i64);
}
```

ASLR

ASLR (Address Space Layout Randomization)

L'ASLR est un mécanisme de sécurité qui consiste à **randomiser l'emplacement en mémoire** des différentes zones d'un processus (image de l'exécutable, piles, tas, bibliothèques, etc.) à chaque exécution.

L'objectif est de rendre les adresses **imprévisibles** afin de compliquer les attaques qui reposent sur la connaissance d'adresses fixes, comme le `return to libc` ou le `Return-Oriented Programming (ROP)`.

Points clés sous Windows

- Introduit à partir de **Windows Vista** (2007).

ASLR

- Activé au niveau du binaire via le flag de liaison `/DYNAMICBASE` (indicateur `IMAGE_DLLCHARACTERISTICS_DYNAMIC_BASE` dans l'en-tête PE).
- Pour les binaires 64 bits, le flag `/HIGHENTROPYVA` augmente l'entropie de la randomisation (adresses sur une plage bien plus large).
- La base des DLL système est tirée au hasard **une fois par démarrage** : une même DLL partage la même adresse entre les processus jusqu'au redémarrage.
- Contournements possibles : **fuite d'adresse** (information leak), pages non relocalisables, ou modules compilés sans `/DYNAMICBASE`.

DEP

DEP (Data Execution Prevention)

Le DEP est un mécanisme de sécurité qui empêche l'**exécution de code depuis des zones mémoire destinées uniquement aux données** (pile, tas, etc.). Une tentative d'exécution dans une telle zone provoque une exception et l'arrêt du processus.

L'objectif est de bloquer les attaques classiques de type **injection de shellcode** où l'attaquant place son code dans une zone de données (ex. la pile) puis détourne le flux d'exécution vers celui-ci.

Points clés sous Windows

- Introduit à partir de **Windows XP SP2** (2004).

DEP

- Repose sur le bit matériel **NX** (No-eXecute) du processeur (`XD` chez Intel) : chaque page mémoire peut être marquée comme **non exécutable**.
- Activé au niveau du binaire via le flag de liaison `/NXCOMPAT` (indicateur `IMAGE_DLLCHARACTERISTICS_NX_COMPAT` dans l'en-tête PE).
- Souvent combiné avec l'**ASLR** : le DEP empêche l'exécution de données, l'ASLR rend les adresses imprévisibles.
- Contournement classique : le **Return-Oriented Programming (ROP)**, qui réutilise du code **déjà exécutable** du programme au lieu d'injecter du nouveau code.

7 Drivers Windows

Composants principaux

Device Driver

Un driver (« pilote ») est un logiciel permettant de s'interfacer avec un périphérique attaché à un ordinateur, comme une imprimante ou une carte graphique.
Les drivers s'exécutent dans le contexte du noyau Windows (« ring 0 »).

Fonctions importantes qui déterminent le comportement du driver :

- `DriverEntry` : représente le point d'entrée du driver (sa fonction « `main` »)
- `DriverUnload` : appelée lorsque le noyau souhaite arrêter le driver

Interactions avec le device driver :

- `Create` et `Close`
- `Read`
- `Write`
- IOCTL : Device Control

DriverEntry : fonctions majeures


```
NTSTATUS DriverEntry(
    _In_ PDRIVER_OBJECT     DriverObj,
    _In_ PUNICODE_STRING    RegistryPath
) {
    // NTSTATUS variable to record success or failure
    NTSTATUS status = STATUS_SUCCESS;

    // Register functions that handles actions on the driver
    DriverObj->DriverUnload = DriverUnload;
    DriverObj->MajorFunction[IRP_MJ_CREATE] = FunCreateClose;
    DriverObj->MajorFunction[IRP_MJ_CLOSE] = FunCreateClose;
    DriverObj->MajorFunction[IRP_MJ_READ] = FunRead;
    DriverObj->MajorFunction[IRP_MJ_WRITE] = FunWrite;
    DriverObj->MajorFunction[IRP_MJ_DEVICE_CONTROL] = FunIoctl;

  // ...
```

DriverEntry : Création du périphérique


```
// ...

  // Create our device object.
  PDEVICE_OBJECT Device = NULL;
  status = IoCreateDevice(
      DriverObject,         // pointer to driver object
      0,                    // device extension size
      &RTL_CONSTANT_STRING(L"\\Device\\MyDriver"),
      FILE_DEVICE_UNKNOWN,  // device type
      0,                    // device characteristics
      TRUE,                 // exclusive
      &Device);             // returned device obj ptr
  if (!NT_SUCCESS(status)) { goto Exit; }

// ...
```

DriverEntry : Création du lien symbolique

Permet d'accéder et d'interagir depuis ce chemin au driver

```
  // ...

  // Create a symlink of the device object for the user
  status = IoCreateSymbolicLink(
  &RTL_CONSTANT_STRING(L"\\DosDevices\\MyDriver"),
  &NtDeviceName
  );
  if (!NT_SUCCESS(status)) { goto Exit; }

  // ...
}
```

Chargement d'un driver

Un driver ne peut être chargé dans le noyau que sous deux conditions:

- Si il est signé par Microsoft, ou une autorité reconnue
- Ou alors, si la vérification est désactivée, en passant Windows en « test mode »

```
# List all services (type=kernel are drivers)
sc.exe query

# Create a new kernel service
sc.exe create my_service `
  type= kernel binPath= c:\path\to\my\driver.sys

# start/stop a service
sc.exe start my_service
sc.exe stop my_service
```

Point de vue utilisateur : ouvrir un device


```
HANDLE hDevice = CreateFileW(
  L"\\\\.\\MyDriver", // drive to open
  0,                  // no access to the drive
  FILE_SHARE_READ |
  FILE_SHARE_WRITE,   // share mode
  NULL,               // default security attributes
  OPEN_EXISTING,      // disposition
  0,                  // file attributes
  NULL                // do not copy file attributes
);

if (hDevice == INVALID_HANDLE_VALUE) {
  // Fail !
}
```

Point de vue utilisateur : interagir

Une fois le handle obtenu, on peut lire, écrire ou appeler des IOCTLs

```
// Trigger read function from driver
ReadFile(hDevice, &buffer, sizeof(buffer), &readen, NULL);

// Trigger IOCTL function 0x1234 from driver
DeviceIoControl(
  hDevice,            // device to be queried
  0x1234,             // operation to perform
  &in, sizeof(in),    // input buffer
  &out, sizeof(out),  // output buffer
  &readen,            // number of bytes returned
  (LPOVERLAPPED) NULL // synchronous I/O
);

CloseHandle(hDevice);
```

ioctl_helper

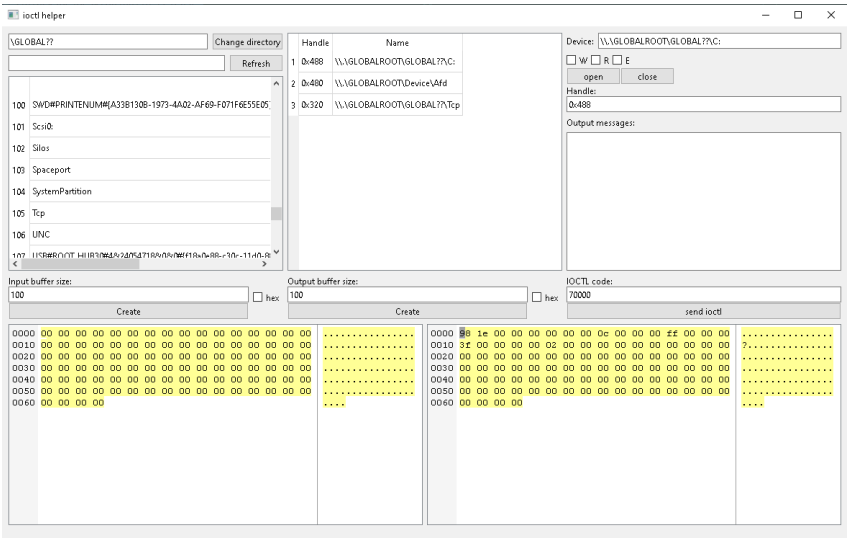


Fig. 20. – github.com/RomanRybachek/ioctl_helper

MiniFilter

- File system filter drivers
- S'attachent au niveau de la gestion logiciel du système de fichier
- Objectifs : surveiller, filtrer, et/ou modifier les opérations I/O du file system
- Anti-virus, chiffrement/ compression à la volée, backup, ...
- Altitude : ordre de passage des filtres (décroissant)

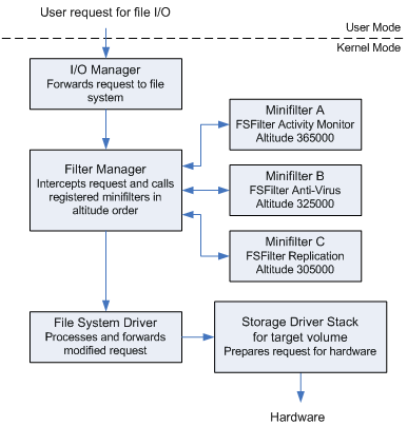


Fig. 21. – I/O stack avec trois minifilters

MiniFilter


```
$ fltmc filters

Filter Name    Num Instances    Altitude    Frame
-------------  -------------  ------------  -----
bindflt                1       409800         0
UCPD                   6       385250.5       0
WdFilter               6       328010         0
storqosflt             0       244000         0
...

$ fltmc load my_minifilter
$ fltmc unload my_minifilter
```

MiniFilter : exemple

Exemple de `DriverEntry` pour MiniFilter :

```
NTSTATUS DriverEntry(
  IN PDRIVER_OBJECT DrvObj, IN PUNICODE_STRING RegistryPath) {
  // ...
  st= FltRegisterFilter(DrvObj, &gFltRegistration, &hMiniflt);
  // ...
  st= FltStartFiltering(hMiniflt);
  if (!NT_SUCCESS(status)) {
      FltUnregisterFilter(hMiniflt);
  }
  // ...
}
```

MiniFilter : exemple

Exemple de `FLT_REGISTRATION` :

```
const FLT_REGISTRATION g_filterRegistration = {
    sizeof(FLT_REGISTRATION),      // Size
    FLT_REGISTRATION_VERSION,      // Version
    0,                             // Flags
    NULL,                          // Context registration
    g_callbacks,                   // Operation callbacks
    InstanceFilterUnloadCallback,  // FilterUnload
    InstanceSetupCallback,         // InstanceSetup
    InstanceQueryTeardownCallback, // InstanceQueryTeardown
    NULL,                          // InstanceTeardownStart
    NULL,                          // InstanceTeardownComplete
    NULL,                          // GenerateFileName
    NULL,                          // GenerateDestFileName
    NULL                           // NormalizeNameComponent
};
```

MiniFilter : exemple

Exemple de `FLT_OPERATION_REGISTRATION` :

```
const FLT_OPERATION_REGISTRATION g_callbacks[] = {
  {
    IRP_MJ_CREATE,       // MajorFunction
    0,                   // FLT_OPERATION_REGISTRATION_FLAGS
    PreOperationCreate,  // PFLT_PRE_OPERATION_CALLBACK
    PostOperationCreate, // PFLT_POST_OPERATION_CALLBACK
  },
  { IRP_MJ_OPERATION_END }
};
```

MiniFilter : exemple

Exemple de callback MiniFilter :

```
FLT_PREOP_CALLBACK_STATUS FLTAPI PreOperationCreate(
  _Inout_ PFLT_CALLBACK_DATA Data,
  _In_ PCFLT_RELATED_OBJECTS FltObjects,
  _Flt_CompletionContext_Outptr_ PVOID* CompletionContext
) {
  // ...
  if (FlagOn(createParamsOptions, FILE_DIRECTORY_FILE)) {
    Data->IoStatus.Status = STATUS_ACCESS_DENIED;
    Data->IoStatus.Information = IO_REPARSE;
    return FLT_PREOP_COMPLETE;
  }

  return FLT_PREOP_SUCCESS_NO_CALLBACK;
}
```

8 Cas d’usage : EDR

Objectifs

- Bloquer un exécutable au runtime
- Scan de la mémoire à la recherche d'indicateurs de compromission
- Scan du système de fichier
  - Nom et chemin
  - Hash
- Scan de comportements suspects
  - Comportement inhabituel d'un processus connu
  - Processus non habituel touchant `LSASS`
- Tourner avec suffisamment de privilèges
  - Avoir les droits de faire toutes les actions ci-dessus
  - Ne pas être désactivé

Processus dans Windows

Process ID

Process ID (PID)

Identifiant unique qui sert à identifier un processus

```
DWORD GetCurrentProcessId();
```

Handles

Handle

Référence à un objet (processus, fichier, ...). Obtenir un Handle incrémente un compteur sur l'objet, prévenant sa destruction tant que tous les Handles n'ont pas été fermés.

Processus dans Windows


```
NTSTATUS status;
HANDLE ProcessHandle;
OBJECT_ATTRIBUTES attr;
CLIENT_ID ClientIds;

InitializeObjectAttributes(&attr, NULL, OBJ_KERNEL_HANDLE,
NULL, NULL);
ClientIds.UniqueProcess = pid;
ClientIds.UniqueThread = 0;

status = ZwOpenProcess(
  &ProcessHandle, PROCESS_ALL_ACCESS,
  &attr, &ClientIds
);

// Do stuff with the handle
ZwClose(ProcessHandle);
```

Processus dans Windows

PEPROCESS

```
typedef struct EPROCESS* PEPROCESS;
```

Pointeur vers un structure « opaque » qui identifie un objet de type Processus dans le noyau.

PEB: Process Environment Block

Structure majoritairement opaque contenant des informations sur un processus. Cette structure est accessible en mode utilisateur (« ring 3 »).

Scan de mémoire

`KeStackAttachProcess`

This routine attaches the current thread to the address space of the target process.

```
KAPC_STATE apcState;
PEPROCESS process = /* retreive the target PEPROCESS */;
KeStackAttachProcess(process, &apcState);

// Access address 0x1234 of the attached process
DbgPrint(
  "Address 0x1234 contains: 0x$llx",
  *(ULONGLONG*)0x1234
);

KeUnstackDetachProcess(&apcState);
// Address 0x1234 of process is not availble anymore
```

NotifyRoutine

- Enregistrement de routines pour :
  - Création de Process : `PsSetCreateProcessNotifyRoutine`
  - Chargement de DLL : `PsSetLoadImageNotifyRoutine`
  - Création de Thread : `PsSetCreateThreadNotifyRoutine`

```
NTSTATUS PsSetCreateProcessNotifyRoutine(
  PCREATE_PROCESS_NOTIFY_ROUTINE NotifyRoutine, // routine ptr
  BOOLEAN Remove // subscribe or unsubscribe from this event
);
```

NotifyRoutine


```
void sCreateProcessNotifyRoutine(
  HANDLE ppid, HANDLE pid, BOOLEAN create
) {
  if (create) {
    PEPROCESS process = NULL;
    PUNICODE_STRING processName = NULL;

    PsLookupProcessByProcessId(pid, &process);
    SeLocateProcessImageName(process, &processName);

    DbgPrint("process created: %d %wZ", pid, processName);
  }
  else {
    DbgPrint("Process %d lost child %d", ppid, pid);
  }
}
```

ObCallback

- Callback pour des opérations sur les processus et threads
- Enregistrement de routines lors de l'ouverture/duplication d'un handle
- Fonction `ObRegisterCallbacks`
- `PreOperation` et `PostOperation` routines

```
NTSTATUS ObRegisterCallbacks(
  [in]  POB_CALLBACK_REGISTRATION CallbackRegistration,
  [out] PVOID                     *RegistrationHandle
);
```

ObCallback


```
typedef struct _OB_CALLBACK_REGISTRATION {
  USHORT                    Version;
  USHORT                    OperationRegistrationCount;
  UNICODE_STRING            Altitude;
  PVOID                     RegistrationContext;
  OB_OPERATION_REGISTRATION *OperationRegistration;
} OB_CALLBACK_REGISTRATION, *POB_CALLBACK_REGISTRATION;

typedef struct _OB_OPERATION_REGISTRATION {
  POBJECT_TYPE                *ObjectType;
  OB_OPERATION                Operations;
  POB_PRE_OPERATION_CALLBACK  PreOperation;
  POB_POST_OPERATION_CALLBACK PostOperation;
} OB_OPERATION_REGISTRATION, *POB_OPERATION_REGISTRATION;
```

ObCallback


```
NTSTATUS status;
HANDLE hRegistration = NULL;
OB_CALLBACK_REGISTRATION callbackReg;
OB_OPERATION_REGISTRATION opReg;

opReg.ObjectType = PsProcessType; // PsThreadType
opReg.Operations = OB_OPERATION_HANDLE_CREATE; // DUPLICATE
opReg.PreOperation = PreOperationCallback;
opReg.PostOperation = NULL;

callbackReg.Version = OB_FLT_REGISTRATION_VERSION;
callbackReg.OperationRegistrationCount = 1;
callbackReg.Altitude = /* ... */;
callbackReg.RegistrationContext = NULL;
callbackReg.OperationRegistration = &opReg;

status = ObRegisterCallbacks(&callbackReg, &hRegistration);
```

9 BYOVD

Une protection ultime

Rappel

Un driver ne peut être chargé dans le noyau que s'il est signé par une autorité de confiance.

- Contexte d'exécution: noyau (ring 0)
- Privilèges maximum: le « graal » pour un malware
- Peu de chances que Microsoft signe un driver malveillant…
- Certification nécessaire avant signature (*Windows Hardware Quality Labs* testing, WHQL)

Comment atteindre ces privilèges ?

Une protection ultime ?

- Malheureusement, certains drivers sont validés par WHQL mais contiennent des vulnérabilités (ou même parfois des trous béants)

⇒ Une vulnérabilité dans un driver légitime peut ouvrir la porte à de l'exécution de code en mode noyau.

https://www.loldrivers.io/

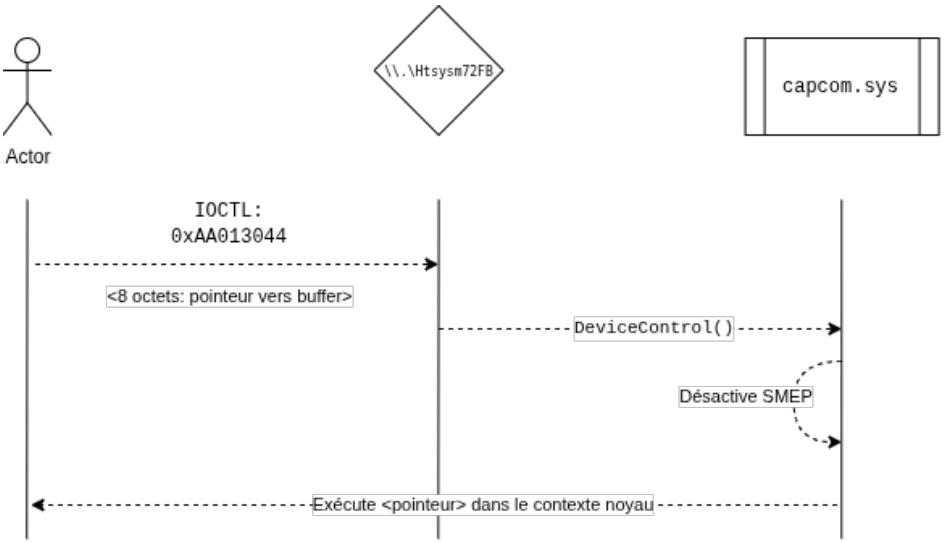
Exemple: `capcom.sys`

- Driver anti-cheat pour Street Fighter V
- Signé par Capcom, validé par Microsoft



- Exécution de code arbitraire
  - Pas de vérification du processus appelant
  - Pas de signature du code
  - Attention, exploit compliqué

Exemple: `capcom.sys`



Reverse de `capcom.sys`

Session de reverse live

Bring Your Own Vulnerable Driver

Définition

Installer délibérément un pilote vulnérable afin de l'exploiter et d'élever ses privilèges

- Prérequis: avoir les droits `SYSTEM` sur la machine
- **But:**
  - Désactiver antivirus, EDR
  - Rootkit
  - Evader la détection
- Utilisé par beaucoup de groupes malveillants[26]

[26] https://attack.mitre.org/techniques/T1068/

10 Patch diffing

Cycle de patchs & 1DAY

Patch Tuesday

Microsoft publie un correctif de sécurité une fois par mois, les premiers mardis sur tous ses produits

Patchs disponibles mais peu d'informations sur les CVEs

CVE	Title	Severity	CVSS	Public	Exploited	Type
CVE-2025-26633	Microsoft Management Console Security Feature Bypass Vulnerability	Important	7	No	Yes	SFB
CVE-2025-24985	Windows Fast FAT File System Driver Remote Code Execution Vulnerability	Important	7.8	No	Yes	RCE
CVE-2025-24984	Windows NTFS Information Disclosure Vulnerability	Important	4.6	No	Yes	Info
CVE-2025-24991	Windows NTFS Information Disclosure Vulnerability	Important	5.5	No	Yes	Info
CVE-2025-24993	Windows NTFS Remote Code Execution Vulnerability	Important	7.8	No	Yes	RCE
CVE-2025-24983	Windows Win32 Kernel Subsystem Elevation of Privilege Vulnerability	Important	7	No	Yes	EoP
CVE-2025-26630	Microsoft Access Remote Code Execution Vulnerability	Important	7.8	Yes	No	RCE
CVE-2025-24057	Microsoft Office Remote Code Execution Vulnerability	Critical	7.8	No	No	RCE
CVE-2025-26645	Remote Desktop Client Remote	Critical	8.8	No	No	RCE

Fig. 24. – Patch Tuesday Mars 2025

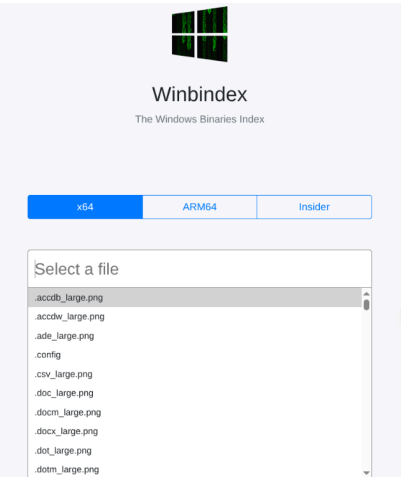


Fig. 25. – winbindex.m417z.com

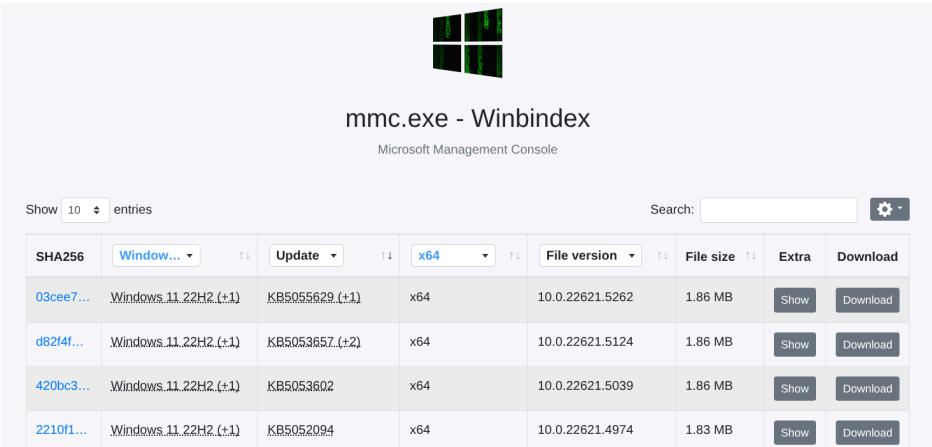


Fig. 26. – Winbindex results for mmc.exe

BinDiff

Programme de diffing de binaires basé sur les bases de données générées par IDA. Génère des scores de similarité sur les fonctions décompilées.

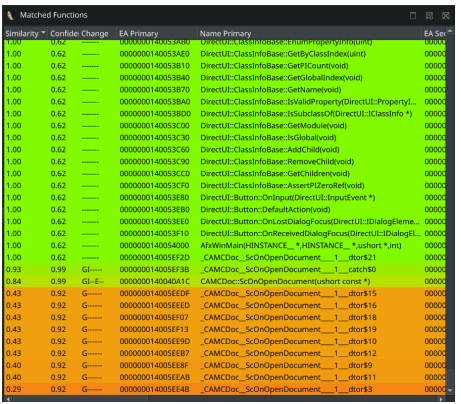


Fig. 27. – BinDiff results for mcc.exe

Le jeu des différences !


```
muipath = ScGetMuiPath(v9, v46, filesource, &v49);

if ( !_IsFileSourceUntrustworthy(filesource) ) {
  // Error MotW
}

if (muipath) {
  ScLoadConsole(v26, v56, muipath, v49, v57, &v52)
} else {
  ScLoadConsole(v26, v56, filepath, v49, v57, &v52)
}
```

```
if ( !_IsFileSourceUntrustworthy(filesource) ) {
  // Error
}

muipath = ScGetMuiPath(v9, v46, filesource, &v49);
if ( !_IsFileSourceUntrustworthy(muipath) ) {
  // Error
}

if (muipath) {
  ScLoadConsole(v26, v56, muipath, v49, v57, &v52)
} else {
  ScLoadConsole(v26, v56, filepath, v49, v57, &v52)
}
```

Fig. 28. – mcc.exe : avant patch (gauche) - après patch (droite)

Petit coup de pouce : `EvaluateCurrentState`


```
if !(EvaluateCurrentState(global_feature_x) {
  muipath = ScGetMuiPath(v9, v46, filesource, &v49);
}

if ( !_IsFileSourceUntrustworthy(filesource) ) {
  // Error
}

if (EvaluateCurrentState(global_feature_x) {
  muipath = ScGetMuiPath(v9, v46, filesource, &v49);
  if ( !_IsFileSourceUntrustworthy(muipath) ) {
    // Error
  }
}

if (muipath) {
// ...
```

Fig. 29. – mcc.exe : après patch avec EvaluteCurrentState

PoC 1DAY !

- CVE-2025-26633
- MotW bypass in mmc.exe
- Laissé en exercice au lecteur…

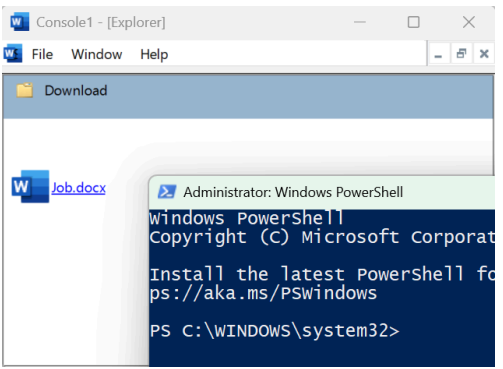


Fig. 30. – Exploitation de CVE-2025-26633 : immitation d'un .docx dans un fichier msc pour lancer une console powershell

11 Conclusion

TP: Solution d'anti-cheat pour demineur.exe

